Pre-Entrega Proyecto Java-BackEnd

Durante estas últimas semanas tuve muy poco tiempo disponible para avanzar en el proyecto, ya que además de trabajar, estoy cursando la Tecnicatura en Desarrollo de Software en el ITFS, donde tuve varios exámenes en estas fechas. Por ese motivo, para poder cumplir con la entrega, decidí partir del proyecto base que vimos en clase el cual adapté alguna de las funcionalidades solicitadas.

Trabajo inicial

Para avanzar de manera ordenada, seguí los siguientes pasos:

- Analicé el proyecto presentado en clase y repliqué su estructura en mi entorno local.
- Organicé el código manteniendo los paquetes com.techlab.ecommerce, tal como se ve en la carpeta src/ del repositorio.
- Revisé el proyecto base presentado en clase, entendiendo cómo estaban organizados los paquetes, las clases y la lógica del servicio.
- Fui adaptando el código, reemplazando partes que necesitaban ajustes para funcionar con mis clases.
- Implementé las funcionalidades solicitadas: registrar, listar, buscar, actualizar y eliminar productos.
- Realicé pruebas manuales para asegurar que la lógica funcionara correctamente.
- Configuré el repositorio Git, resolviendo conflictos iniciales y subí el proyecto completo.

Mejoras en la estructura del menú (MenuProducto)

En el proyecto original, la visualización del menú y la ejecución de cada opción estaban dentro de Main.java, Para mejorar la estructura, trasladé toda la lógica del menú a la clase MenuProducto.

```
Run | Debug
public static void main(String[] args) {

    // Dependencias principales
    Scanner sc = new Scanner(System.in);
    ProductoService service = new ProductoService();

    // Carga inicial opcional
    cargarDatosDePrueba(service);

    // Inicia el menú principal
    MenuProducto menu = new MenuProducto(sc, service);
    menu.iniciar();

    sc.close();
}
```

Ahora, MenuProducto:

- muestra el menú completo,
- gestiona la lectura de la opción ingresada por el usuario,
- y ejecuta directamente las acciones correspondientes (agregar, listar, buscar, actualizar o eliminar productos).

Debido a este cambio Main.java se limita únicamente a iniciar el programa, mientras que MenuProducto concentra toda la interacción con el usuario.

```java
// Bucle principal del menú
public void iniciar() {
    int opcion;

    do {
        mostrarMenu();
        opcion = Validador.leerEntero(sc, mensaje: "Elija una opción: ");

        try {
            switch (opcion) {
                case 1 -> agregarProducto();      // Alta
                case 2 -> listarProductos();      // Listado
                case 3 -> buscarProducto();       // Búsqueda por ID
                case 4 -> actualizarProducto();   // Modificación
                case 5 -> eliminarProducto();     // Baja
                case 6 -> System.out.println(x: "¡Hasta luego!");
                default -> System.out.println(x: "Opción inválida.");
            }
        } catch (Exception e) {
            // Muestra cualquier error de validación o negocio
            System.out.println("⚠ " + e.getMessage());
        }

        System.out.println();

    } while (opcion != 6);
}
```

```java
// Muestra el menú principal
public void mostrarMenu() {
    System.out.println(x: "\n--- TechLab --- Gestión de Productos ---");
    System.out.println(x: "1. Agregar producto");
    System.out.println(x: "2. Listar productos");
    System.out.println(x: "3. Buscar producto");
    System.out.println(x: "4. Actualizar producto");
    System.out.println(x: "5. Eliminar producto");
    System.out.println(x: "6. Salir");
}
```

Mejoras y validaciones implementadas

Durante el desarrollo identifiqué que el sistema original realizaba las validaciones recién al final del proceso, cuando se intentaba guardar el producto. Por ejemplo:

- si el nombre estaba vacío,
- si el stock era menor que 0,
- si el precio era menor o igual que 0,

el sistema mostraba el error recién al final, pero no volvía a pedir los datos, lo que obligaba a repetir todo el ingreso.

Para mejorar esto implementé validaciones inmediatas para cada atributo:

- nombre obligatorio (no vacío),
- precio mayor que 0,
- stock no negativo,
- descripción obligatoria,
- categoría válida,
- y cualquier otro atributo relevante del producto.

```java
// Lectura con validación
public static int leerEntero(Scanner sc, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String input = sc.nextLine().trim();

        if (input.isEmpty()) {
            System.out.println(x: "Debe ingresar un número entero. No puede quedar vacío
            continue;
        }

        try {
            int valor = Integer.parseInt(input);

            if (valor < 0) {
                System.out.println(x: "El stock no puede ser negativo.");
                continue;
            }

            return valor;

        } catch (NumberFormatException e) {
            System.out.println(x: "Debe ingresar un número entero válido.");
        }
    }
}
```

```java
public static double leerDouble(Scanner sc, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String input = sc.nextLine().trim();

        if (input.isEmpty()) {
            System.out.println(x: "Debe ingresar un número. No puede quedar vacío.");
            continue;
        }

        try {
            double valor = Double.parseDouble(input);

            if (valor <= 0) {
                System.out.println(x: "El precio debe ser mayor a 0.");
                continue;
            }

            return valor;

        } catch (NumberFormatException e) {
            System.out.println(x: "Debe ingresar un número válido.");
        }
    }
}
```

```java
// Permite vacío (para actualizar)
public static String leerTexto(Scanner sc, String mensaje) {
    System.out.print(mensaje);
    return sc.nextLine();
}

// NO permite vacío (para agregar)
public static String leerTextoObligatorio(Scanner sc, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String input = sc.nextLine().trim();

        if (!input.isEmpty()) {
            return input;
        }

        System.out.println(x: "Debe ingresar un texto. No puede quedar vacío.");
    }
}
```

Ahora, si el usuario ingresa un dato inválido, el sistema lo detecta en el momento y vuelve a solicitarlo, evitando errores posteriores.

Reorganización en ProductoService y agregado del rubro

En la clase ProductoService realicé una reorganización general para mejorar la claridad y la tarea de cada método. En el proyecto original, algunos métodos estaban mezclados o no seguían un orden lógico, lo que dificultaba su mantenimiento. Para mejorar esto, reorganicé la clase siguiendo la siguiente secuencia:

- primero los métodos de guardar productos,
- luego los de listado y búsqueda,
- después los de actualización,
- y finalmente los de eliminación.

```java
// Guarda un producto nuevo después de validar sus datos
public Producto guardar(Producto p) {
    Validador.validarNombre(p.getNombre());
    Validador.validarPrecio(p.getPrecio());
    Validador.validarStock(p.getStock());
    Validador.validarCategoria(p.getCategoria());
    Validador.validarRubro(p.getRubro());

    p.setId(contadorId++);   // Asigna ID único
    productos.add(p);        // Lo agrega a la lista
    return p;
}

// Devuelve todos los productos cargados
public List<Producto> listarTodos() {
    return productos;
}

// Busca un producto por ID; si no existe, lanza excepción
public Producto obtenerPorId(int id) {
    for (Producto p : productos) {
        if (p.getId() == id) {
            return p;
        }
    }
    throw new ProductoNoEncontradoException("No se encontró un producto con id " + id);
}
```

```java
// Actualiza un producto existente reemplazando todos sus campos
public Producto actualizar(int id, Producto datos) {
    Producto p = obtenerPorId(id); // Busca el producto original

    // Valida los nuevos datos
    Validador.validarNombre(datos.getNombre());
    Validador.validarPrecio(datos.getPrecio());
    Validador.validarStock(datos.getStock());
    Validador.validarCategoria(datos.getCategoria());
    Validador.validarRubro(datos.getRubro());

    // Aplica los cambios
    p.setNombre(datos.getNombre());
    p.setPrecio(datos.getPrecio());
    p.setStock(datos.getStock());
    p.setCategoria(datos.getCategoria());
    p.setRubro(datos.getRubro());

    return p;
}

// Elimina un producto por ID
public void eliminar(int id) {
    Producto p = obtenerPorId(id); // Si no existe, lanza excepción
    productos.remove(p);
}
```

También ajusté la lógica interna para que cada método cumpla una única función y completé el uso de validaciones antes de modificar la lista de productos.

Además, incorporé el nuevo campo rubro, que funciona como una categoría general que complementa a la categoría específica del producto. Este atributo permite clasificar los productos de manera más amplia (por ejemplo: Almacén, Higiene, Lácteos), mientras que la categoría mantiene el detalle específico. Para integrarlo correctamente, actualicé:

- el constructor de Producto,
- los getters y setters,
- el método toString(),
- y los métodos de alta y actualización dentro de Menu Producto.

```java
// Atributos del producto
private int id;             // Identificador único asignado por el sistema
private String nombre;      // Nombre del producto
private double precio;      // Precio unitario
private int stock;          // Cantidad disponible
private String categoria;   // Categoría general (ej: Bebidas, Almacén)
private String rubro;       // Rubro específico (ej: Almacén y bebidas)

// Constructor para crear un producto nuevo
public Producto(String nombre, double precio, int stock, String categoria, String rubro) {
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
    this.categoria = categoria;
    this.rubro = rubro;
}
```

```java
public String getRubro() { return rubro; }
public void setRubro(String rubro) { this.rubro = rubro; }

// Pasaje a texto para mostrar el producto
@Override
public String toString() {
    return "ID: " + id +
           " | " + nombre +
           " | $" + precio +
           " | Stock: " + stock +
           " | Cat: " + categoria +
           " | Rubro: " + rubro;
}
```

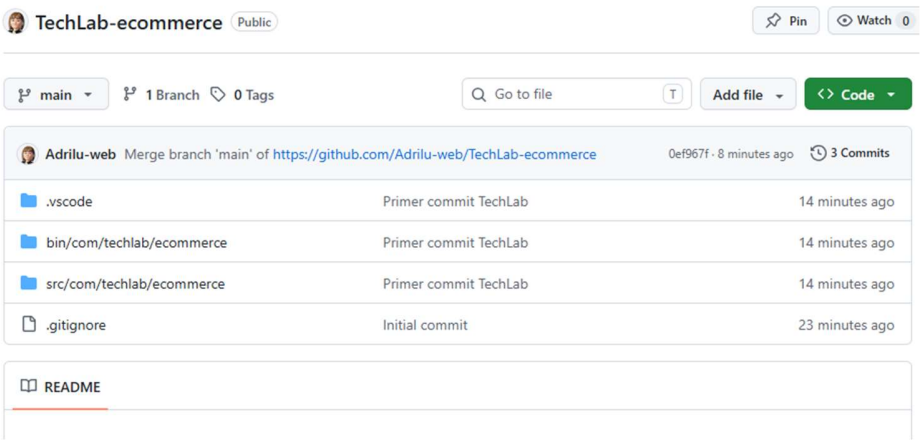
Dificultades y cómo las resolví

- Falta de tiempo, que me llevó a priorizar las funcionalidades principales.
- Conflictos con Git, resueltos mediante un merge y la configuración del repositorio.
- Adaptación del código base, ajustando métodos para que funcionaran con mi implementación.
- Validaciones mejoradas para facilitar el ingreso de datos al usuario.

Repositorio en GitHub

Mi trabajo quedó registrado en el repositorio público de GitHub: **TechLab-ecommerce ,**
donde pueden verse los commits principales (Initial commit, Primer commit TechLab y el merge inicial), así como la estructura final del proyecto en las carpetas src/com/techlab/ecommerce y bin/com/techlab/ecommerce.

En el quedo organizado de la siguiente manera:



Conclusión

En resumen, tomé como base el proyecto presentado en clase, lo adapté y mejoré, incorporando las funcionalidades solicitadas, agregando validaciones dejando el código funcional y versionado en GitHub.